



A FIELD REPORT FROM INSIDE THE SHIFT

Fábio Ramos, CEO
Jônadas Techio, Staff PM

Axur

Manifesto: *Restructuring
Knowledge Work for
Human-Agent Teams*

Field report from a 23-person internal pilot at Axur

By Fábio Ramos, CEO, and Jônadas Techio
Axur · May 2026

This document is the companion to *Restructuring Knowledge Work for Human-Agent Teams*. The manifesto makes the argument; this is the substrate behind it. The architecture, the conventions, the failure modes, and the patterns from seven weeks of practice with more than twenty contributors across eight functions.

We are not the first to make this kind of bet. [Block](#) has described an adjacent architecture at company scale. [Beacraft](#) has named the role hierarchy that emerges. [Zapier](#) has documented the pattern across functions. This is reporting from team scale.

We ran our pilot primarily on Claude, but the architecture is tool-agnostic by design. The same files work with Cursor, OpenCode, GitHub Copilot, AntiGravity, or any agent that reads Markdown.

If you are evaluating this approach for your own company, the closing section, *If you want to try this*, is what we would have wanted at week one.

How this report maps to the manifesto

This report is the substrate behind the manifesto. The six principles in the manifesto's *Six rules underneath the architecture* map one-to-one to this report's section of the same name and to *Inside the substrate*. The manifesto's three observations in *What the rules enable* are grounded here: *The org chart becomes skeuomorphic* earns its place from *Where the architects came from*, *How the pattern spreads*, and *From individual wins to team-level projects*. *Agents contribute arguments, not just artifacts* is grounded in *When the agent argued back*. *Tokens are the new working capital* is grounded in the section of the same name. The corollary that *everyone should use Git* is grounded inline in principle 1 of *Six rules underneath the architecture* and in *Git as the toll gate*, inside *What broke*. If you arrived here from the manifesto looking for the receipts, this is the map.

Six rules underneath the architecture

Six principles shaped the system. We did not define them upfront. They emerged from the constraints we encountered, and we formalized them after the fact. They orient everything that follows in this document.

1. Knowledge is Markdown. The canonical format for everything the company knows is plain Markdown files in a Git repository. Not a database, not a SaaS platform, not a proprietary format. Markdown is human-readable, version-controlled, portable across tools, and natively parseable by every major LLM. If a piece of knowledge cannot be expressed as a Markdown file, it either needs to be restructured or it belongs somewhere else. The constraint is productive: it forces clarity on what counts as durable knowledge versus what counts as ephemera.

2. Progressive disclosure. No agent and no human should need to load the entire knowledge base to answer a question. The system is organized as a thin index at the top, a root instruction file of about fifty lines, that points to topic-specific directories, each with its own focused files. The agent reads the index first, then loads only what the current task requires. Anthropic's [own engineering guidance on context for AI agents](#) recommends the same pattern: present ideas at "the right altitude," favor concise high-signal tokens, and let agents incrementally discover relevant context through exploration. A

2025 [LangChain study](#) confirmed it empirically: a concise, structured manifest consistently outperformed wiring in raw documentation. The corollary worth naming explicitly: the human does not need to know where each piece of information lives. Asking the agent "where is X?" or "which file should I update?" is the model's job, not the user's. The structure exists for the agent to navigate; the natural-language interface exists for the human. This is one of the harder shifts for new contributors to internalize, because every prior file system trained them to know where things are.

3. Shared truth, personal workspace. There is a single source of truth that everyone reads from and contributes to. There is also a personal layer: each contributor's working notes, task lists, and preferences, private by default. The shared layer is governed (reviewed before merge). The personal layer is autonomous (each person manages their own). This separation prevents the two failure modes of knowledge systems: the wiki nobody updates because contributing is too heavy, and the wiki nobody trusts because anyone can write anything. The first cross-author merge we did, between the CEO's RFP-oriented product perspective and a PM's strategy-oriented perspective on the same products, tested this directly. Both perspectives survived. Conflicts on shared metrics forced explicit decisions about which version was authoritative. Neither author had to abandon their layer to contribute to the other.

4. Tool agnosticism. The knowledge base does not depend on any specific AI tool, editor, or platform. Git is the transport layer. Markdown is the storage format. Any LLM agent that can read files and follow instructions can operate on this substrate. Tool-specific configuration exists as a thin adapter on top of the portable base. We ran our own pilot primarily on Claude, but contributors using Cursor, GitHub Copilot, OpenCode, and AntiGravity adapted the same repository to their existing tools without modification. One contributor rotated through four different agentic tools in a single working day, choosing each for its strengths and rotating between them to manage token limits. The knowledge base was indifferent to all of them.

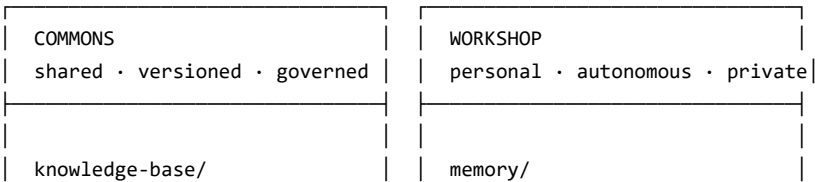
5. Status markers as behavioral gates. Documents contain machine-readable signals about their own reliability. [TBD] means the agent should not present adjacent content as fact. [NEEDS-UPDATE] means flag uncertainty. [DRAFT] means do not appear in final outputs. [CONFIDENTIAL] means extra care. These markers turn passive documents into active instructions: constraints that travel with the knowledge, not with the tool configuration. Once you have markers, the document is no longer a description of state. It is a participant in the workflow.

6. Skills as reusable intent. Agent workflows should be encoded as reusable skills rather than one-off prompts. A skill is a Markdown file that defines a specific workflow, its trigger, the tools it should use, and its expected output. Skills are version-controlled alongside the knowledge base, shareable across contributors, and composable with each other. A skill for generating a quarterly business review can call a skill for applying brand identity, which can call live data connectors, producing a complete branded document from a single natural-language command. This is what transforms the knowledge base from a reference library into an executable operating system.

Inside the substrate

The substrate has two conceptual layers and five concrete surfaces in the file system. Each surface has its own change rate, governance, and lifecycle.

THE SUBSTRATE: TWO LAYERS, FIVE SURFACES



what the company knows (stable reference)	people, projects, drafts, glossary, story bank
shared-projects/ work in flight (active collaboration)	CLAUDE.md + TASKS.md personal navigation and working memory
tools/ executable capabilities (scripts, MCP servers)	.claude/skills/ procedural workflows (some shared via plugins)
slow change, reviewed before merge	fast change, autonomous, gitignored from shared repo

In short: knowledge-base/ is what the company knows. shared-projects/ is work in flight. tools/ is executable code. memory/ is personal context, subdivided into people, projects, drafts, glossary, and story bank. CLAUDE.md plus TASKS.md plus .claude/skills/ is personal navigation and procedural knowledge.

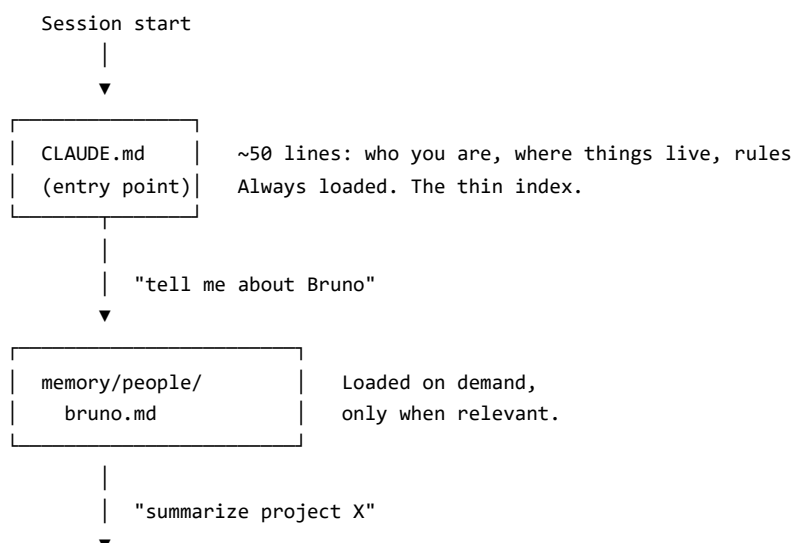
The deeper observation is that the separation does the heavy lifting. Personal memory inside the shared knowledge base would either pollute the shared base (too much subjective context) or under-share the personal one (too risky to commit). The split lets each layer optimize for its job.

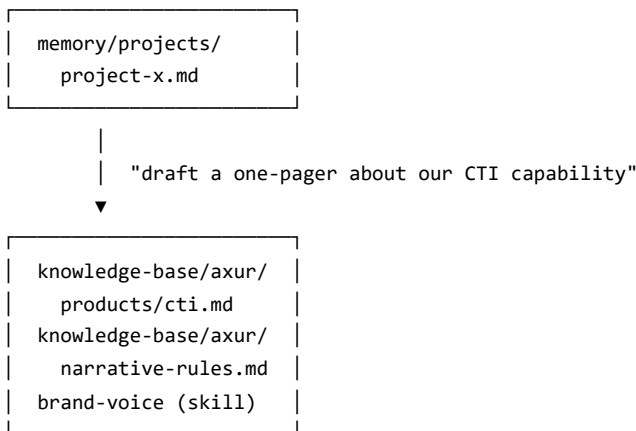
The five-surface anatomy emerged in the first ten days, not by design but by friction. The instruction manifest at the workspace root started at hundreds of lines and contained inline tables of people, glossary, and project context. Every session loaded thousands of tokens of mostly-irrelevant context before any real work could begin. Refactoring it down to roughly fifty lines, with the people, glossary, and project tables moving into their own files under memory/, halved load time and improved relevance immediately. Within a week, three other contributors had reproduced the same pattern in their own workspaces without being told to.

The same structure that lets relevance win — load this project's context, ignore the rest — also gives a coarse handle on segregation. Sensitive content can live in its own project folder, kept out of sessions where it does not belong. That handle is real but limited; we return to its limits in *What we don't yet know*.

The architecture is half the story. The other half is how the agent traverses it.

PROGRESSIVE DISCLOSURE — what the agent loads, when





The agent never loads the full repo. It loads what the current question needs, and nothing more.

The repository can grow to thousands of files without slowing the agent down, because at any given moment the agent operates on a small, relevant subset. The substrate is large; the working set is small.

The conventions that keep it honest

Conventions, in this context, are not bureaucracy. They are the small pieces of structured intent (markers, manifests, status tags) that travel with the data and let the agent operate against the substrate without supervision on every call. There are seven that earned their place.

Status markers as behavioral gates. Any document can carry inline markers that the agent reads as instructions, not as decoration. [TBD] means the value is unknown — do not present as fact. [NEEDS-UPDATE] means the content may be outdated — flag uncertainty when citing. [DRAFT] means work in progress — do not use in final outputs. [CONFIDENTIAL] means internal use only — extra care required. The agent treats these markers as gates that change its behavior. The constraints travel with the knowledge, not with the chat session.

Instruction manifests. Every workspace has a thin manifest at its root: CLAUDE.md for Claude-based tools, AGENTS.md for the cross-tool open standard, .cursorrules for Cursor. The manifest is the entry point: about fifty lines, pointing the agent to where shared knowledge lives, where personal context lives, what behavioral rules apply, and what status markers mean. The manifest does not contain knowledge itself; it contains navigation. Knowledge lives in dedicated files loaded on demand.

Agent-addressed documents. There is a class of document that did not exist before LLMs: the document written for the agent to follow, never read by a human. Our setup-interview.md is one. It is a Claude instruction script that runs an onboarding conversation with a new contributor and generates their personal context files. The human never reads it. They say "set me up", and the agent does the rest. Once you start writing for the agent as a first-class reader, document conventions change. No banners. No illustrations. Imperative verbs. Explicit instructions about what files to create and what fields to populate.

Recursive learning loops. Every session ends with a session-review skill that updates the contributor's memory files based on what happened during the session. New facts about people get added. Project context gets refreshed. Decisions get logged. The next session begins by reading the updated memory and operates on better context than the previous session had. The agent compounds across sessions, not because the model has memory, but because the workspace does.

Skills as commons. Skills start in `.claude/skills/` in personal workspaces. When a skill becomes useful to others, it gets promoted into a plugin in the shared repository and distributed across the org. The brand-voice skill is the canonical example: it began as personal, became a plugin, and is now auto-enabled in every contributor's environment, applying the company's voice to every output the agent produces, with no one needing to know it is running. Skills are infrastructure, and infrastructure spreads.

Schema-level prompt engineering. When the agent queries a data source through MCP, the quality of the result depends heavily on how well the data layer documents itself. A model receiving ambiguous field names will attempt multiple queries through trial and error, consume tokens, add latency, and produce inconsistent results. The fix is to embed hints in the data layer: human-readable descriptions of what each field means, what valid values look like, and how to interpret the schema. We started calling this *schema-level prompt engineering*. It is the data-layer cousin of behavioral gates in the knowledge layer. Same insight, one altitude lower.

Branch-per-default. When the team grew past five active contributors, we hit predictable conflicts on the main branch. During a workshop demo, two contributors committed to main simultaneously and produced an instructive merge conflict — exactly the scenario the next rule resolved. From then on: branch-per-default for any non-trivial change. Trivial single-file edits (typo fixes, date updates) can still go directly to main. The rule lives in the workspace manifest so the agent enforces it during commit operations.

These seven did not exist on day one. They emerged as failure modes appeared, got formalized, and stuck because removing any one of them brought the failures back.

Where the architects came from

The framework that names what we watched is from Beacraft: in agentic organizations, Operators do work with AI assistance, Designers build reusable workflows, and Architects encode intent and judgment into infrastructure that agents navigate autonomously. The discourse around this hierarchy assumes Architects emerge slowly, as a specialized role, mostly inside engineering. In our pilot, with the substrate underneath, several team members made that shift in days. None of them were engineers. What follows are six examples of what they built, why each mattered, and what the pattern across them means.

A product manager builds the first plugin.

The first plugin distributed across the company came out of the product team, not engineering. A product manager identified six analytical questions that the company's analysts were answering manually, day after day, against the same underlying data. He wrote a skill for each: credential exposure assessment, phishing campaign mapping, threat actor profiling, dark web exposure monitoring, threat landscape briefing, and supply chain risk assessment. Each skill encoded the methodology — what data sources to query, how to rank results by severity, what to include in the final report — as a Markdown file with a thin metadata header. The skills were initially fed manual data; later, when the company shipped MCP connectors to its threat intelligence platform, the skills were wired into the live data layer the whole organization queries. None of this required engineering work. The skills were composed in natural language. The MCP integration came later, but the methodology was preserved unchanged. The same product manager later built another skill: an orchestrator that took thousands of external domains, ran them through five intelligence layers in parallel, ranked them by risk, and produced a complete branded report. What had been a multi-week analyst job became an overnight job.

Marketing builds the first auto-enabled plugin.

The marketing team built the first skill that runs invisibly. The brand-voice plugin reads every output the agent generates and applies the company's voice — vocabulary rules, tone constraints, capitalization conventions, microcopy preferences. The user does not invoke the plugin. They ask the agent to write something. The output comes back already in voice. The

plugin is auto-enabled across every contributor's environment. The marketing team did not write code. They wrote guidelines as Markdown, structured as the plugin format requires, and committed them to the shared repository. The first version landed within a week of the team joining the pilot.

Customer success builds recurring automations.

A customer success contributor with no prior experience in agentic IDEs built two recurring workflows in his first weeks. The first is a daily morning briefing that runs at 8am: it pulls the day's calendar, scans new emails for action items, reads relevant Slack channels, and produces a prioritized action plan. The second is a pre-meeting prep that fires thirty minutes before any client call: it surfaces the account's recent activity, pending items, last touchpoint, and likely topics. Both were configured in natural language. Both run as scheduled tasks against the shared substrate. Neither required ongoing maintenance.

Customer success builds a proposal pipeline.

The same contributor built a different kind of skill that crosses a function boundary: a request-for-proposal converter. The input is a client's RFP plus the team's internal commercial proposal draft. The output is a fifteen-page branded document, in voice, with the right structure for the customer's procurement process. The skill produced a polished proposal for a real client in minutes. This is a function boundary that used to live between customer success and pre-sales. The substrate dissolved it.

Operations builds an investigation workflow.

The operational analyst team built a skill that takes a case ID, retrieves the case and its attachments from the company's ticketing system, runs multiple data connectors against the artifacts (credentials, dark web, phishing), and produces a formatted investigation report. The skill orchestrates four MCPs and applies brand identity. What was previously a multi-step manual workflow with handoffs across analysts became a single command. The team that built this is not an engineering team.

Intelligence improves the foundations.

While most contributors built skills (visible artifacts), at least one built something that improved every other skill. An intelligence analyst spent time documenting the schema of the company's threat intelligence platform: field names, valid values, common pitfalls, recommended query patterns. The result was not a skill anyone runs directly. It was schema-level prompt engineering — documentation embedded in the data layer that travels with the data. After the documentation landed, every other workflow that queried the same data became more reliable. Token consumption per query dropped. Query consistency across sessions improved. The visible artifact of this work is hard to point at. The compounding effect is impossible to miss.

Six examples, six functions, none of them engineering, all of them building infrastructure that other people use. The pattern across them is consistent. The contributor identifies a workflow they currently do manually, repeatedly. They describe it to the agent in natural language. The agent helps them structure it as a Markdown file with a thin metadata header. They commit it to the shared repository, sometimes through a branch and PR, sometimes directly. Other contributors discover it through the index and start using it. Some adapt it.

This pattern matches what Zapier's [AI Fluency Rubric](#) describes as the Transformative tier across six functions: not teams that work faster, but teams that have re-engineered how work happens, with categories of work that no longer require human involvement and frameworks that others adopt. Our pilot produced this across multiple functions without executive mandate. The architecture made it cheaper to encode expertise than to merely document it.

The Operator-to-Architect transition is described in the agentic-organizations literature as a slow, specialized role change. Inside our pilot, with the right substrate underneath, it happened in days for several team members, on different paths, with different outputs. The tooling did not make this happen. The substrate did.

How the pattern spreads

The Operator-to-Architect shift does not happen evenly across a team. In each function we onboarded, one contributor tried the substrate first, hit the friction, built something useful, and reported back. The rest of the function watched. Once the first person's productivity was visibly different — a daily briefing that runs on its own, an investigation report that took hours and now takes minutes, a deck that comes out already in voice — the second and third contributors joined, often within the same week.

We started calling these first contributors evangelists, with no implication beyond the practical fact that they go first. They are not always the most senior person in the function, and they are rarely the most cautious. They share one trait: a tolerance for tinkering with their own tools. The majority of the team is not made of evangelists, and that is fine. The majority needs to see the benefit on a colleague's screen before investing time. The evangelists give them that screen.

The metaphor we keep returning to is the electric bicycle. The evangelist is still pedaling and steering — the work is theirs — but they have an order of magnitude more range and force than the colleague pedaling next to them. That visible difference is what recruits the next contributor. It is also why "start with two people" is the cheapest possible advice for anyone replicating this. Two evangelists will pull a function. Twenty mandated participants will not.

By week seven, the pattern had repeated across functions in parallel. Customer success had multiple recurring automations in active development by different contributors. Internal research had skills already in production and others mid-build. Marketing's first plugin had seeded a second plugin in the same team. Operations had spawned a lineage of investigation workflows now reaching adjacent intelligence functions. The six examples above are not the full inventory. They are the seed of each lineage. We started two contributors. We watched eight functions germinate.

The role of the evangelist evolves once the first win exists. The same person who explored the substrate alone in week one is now the most natural person to pull the rest of their function up — not by writing another skill themselves, but by helping the next two or three colleagues build theirs, by reviewing what their function commits to the shared repository, by pointing out which of the function's recurring tasks should become skills next. The transition is real. It is also worth designing for, because it does not happen automatically. Once the first win exists, the evangelist's job is no longer to keep going alone.

When the agent argued back

Speed is not the most striking thing about this. The most striking thing is the kind of work the system began to do. Across the seven weeks, the agent stopped acting purely as an executor and began acting as a participant who had read more than anyone else in the room. There are four cases worth describing in detail.

The leadership-meeting reframing.

Earlier in the experiment, a leadership meeting produced decisions that needed immediate operationalization, including a phased external commitment to a partner organization. The meeting also surfaced an unresolved technical debate: two approaches to extending the product had been framed internally as competing alternatives. The agent processed the transcript, cross-referenced it against the shared knowledge base, ran market research on its own initiative, and concluded that the framing was wrong. The two approaches were already deployed together across the ecosystem, complementary by design, with documented usage patterns at scale. The corrected framing reshaped the executive briefing from presenting a choice to presenting a sequence. Transcript to revised executive briefing, in one working session. The agent did not just execute. It contributed an argument that changed the outcome.

The strategic plan first draft.

A few weeks later, a contributor asked the agent to draft a strategic plan for a forward-looking program with multiple

stakeholders and a tight timeline. The instruction was: read the relevant API documentation, cross-reference it against the shared knowledge base, and produce a plan. The output was a plan the team described as *very good for a first draft*. The bar for *first draft* had moved up. The agent was not faster than a human at producing strategic plans. It was producing a plan with the breadth of context a human plan would normally take a week of meetings to gather — because the substrate already contained the relevant decisions, constraints, prior conversations, and stakeholder profiles, and the agent could read all of them in one pass.

The schema ambiguity surfaced by behavior.

An intelligence analyst tested an MCP connector with a structured forensic prompt. The agent's behavior across runs was inconsistent: it sometimes used one schema field and sometimes used another for what looked like the same query. The inconsistency was the signal. The schema was not wrong; it was under-specified. The agent did not solve the problem. It revealed it. Once revealed, the fix took an afternoon: documentation embedded in the data layer that traveled with the data. Without the agent's behavior, we would have been losing query consistency for weeks before noticing. *Schema-level prompt engineering* exists as a convention in this report because of cases like this one.

The corrected attributions in meeting notes.

The smallest case is also the most everyday. A contributor asked the agent to extract action items from notes a colleague had typed up. The agent identified that the notes had attributed two decisions to the wrong people and proposed surgical edits across the affected memory files to correct the record. The contributor accepted the edits. This is the cheapest version of the pattern. It happens often enough that it has stopped being remarkable.

The pattern across these is not that the agent replaced human judgment. It is that the agent participated in the work as someone who had read more than anyone else in the room. It does not always argue, and when it argues, it is not always right. When it is right, it is sometimes obvious in the moment and sometimes obvious only after the team accepts the change and notices, two weeks later, that no one is going back to the original framing.

This is the property of the architecture that gets predicted least often and matters most. Operators with substrate underneath them do work faster. Architects with substrate underneath them encode methodology so it compounds. The substrate also turns the agent into a contributor in its own right. The next phase of the experiment is whether we can design for that without losing trust in human judgment.

From individual wins to team-level projects

The architecture's first phase tested whether the substrate could carry individual contributions. Six functions answered yes. The second phase asks a different question: whether the substrate can carry collective change — whether a function as a whole can operate differently because of what was built into it. This is the operational test of the manifesto's claim that roles become fluid when the substrate exists. Not just that one Architect emerges in each function, but that the function as a whole reshapes around what the Architect built.

Each of the functions named in the previous section has now committed to a project that does exactly that. The project is proposed by the function, not by leadership. It is described in language the function uses for its own work. It is measured by changes the function values, not by metrics imposed from outside. The evangelist of the function is the project's owner, not because of seniority, but because they are the person who has already lived inside the substrate.

Three projects are illustrative.

Customer success is targeting hyperproductivity for the existing book and, with that headroom, extending the high-touch attention currently reserved for the largest accounts to the rest of the portfolio. The bet is that the same skills which already automate the morning briefing and the pre-meeting prep can absorb enough of the recurring account work that a

CS contributor handles several times the accounts at the same level of attention. If it works, the company stops paying the implicit tax of segmenting which clients get a real human and which get triage. The function changes from a tiered service model to a uniformly high-touch one, with the substrate carrying the difference.

Investigations is targeting a state of revision rather than original construction. Today, an investigator reads a case, queries multiple data sources, builds a dossier from scratch, formats the result, and hands it off. The project is to push as much of that pipeline upstream as the substrate will carry: the agent reads the case, runs the queries, drafts the dossier in the right format already in voice, and the investigator's role becomes critique and judgment rather than assembly. The artisanal share of the work falls; the share that requires human discernment stays. The function changes from constructing dossiers to ratifying them.

Marketing is targeting end-to-end content autonomy across the company. The brand-voice plugin already ensures every output the agent produces comes back in voice. The next step is the rest of the chain: any contributor in any function can produce branded, formatted, on-message content from a natural-language ask, without routing through marketing for review. Marketing's role shifts from gatekeeper of brand expression to designer of the substrate that carries the brand. This is the most explicit case of the underlying pattern: a function building infrastructure that lets the rest of the company do the function's old work without the function's old bottleneck.

What is intentional in this design is the ownership. The projects are not pilots delegated to consultants or task forces. They are committed by the functions that will live with the result. The substrate makes this feasible because the methodology of each function is now encoded as skills and shared knowledge, not as institutional memory in the heads of three senior people. Skills are what compose. They compose only if the function is the one composing them.

The next chapter of the experiment is whether this scales. We have evidence that the substrate carries individual change. We have hypotheses, now committed publicly across the functions named above, about what it carries collectively. We will know more by week fourteen than we know now.

From files to live data

The architecture so far describes the static substrate: files in a Git repository, read by the agent on demand. The MCP layer adds the dynamic surface: live data queryable in natural language.

The Model Context Protocol is an open standard that lets any compatible AI agent access external systems through structured tool calls. MCP servers expose data sources as queryable tools rather than documents. Where Markdown files encode what the organization knows in a stable way, MCP servers expose what the organization's systems contain in real time.

We built five connectors during the pilot, each exposing a specific data layer of our threat intelligence platform: leaked credentials, monitored domains, dark web messages, ticket management, and threat intelligence bulletins. The connectors run as separate processes and expose schemas the agent can query in natural language.

The skill layer and the data layer are deliberately separated. Skills (the methodology layer) are version-controlled Markdown files in the shared repository. MCP connectors (the data layer) are running services. Skills can be built and tested without MCP connections, using manual data as fallback. MCP connections enrich skills as they come online. This separation lets each layer evolve independently.

The most useful insight from the MCP work is the one captured in the conventions section: the richer the underlying data model, the harder it is to make queries consistent. Schema-level prompt engineering — documentation embedded in the

data layer that travels with the data — was the fix. Same insight as behavioral gates in the knowledge layer, one altitude lower.

Until recently, using MCP connectors required a CLI environment (Claude Code or equivalent terminal-based tooling). Contributors comfortable in a terminal could query the live data layer; contributors using the desktop interface could not. This was the architecture's most visible inequality.

That problem is now solved. We packaged the connectors as a `.mcpb` desktop extension, installable directly inside the desktop interface, with no terminal access required. The full data layer is available to every contributor in the pilot. External customers are now testing the same packaging in production. The next step is publishing the connectors to the agent vendor's plugin marketplace, so any organization with the right credentials can install them with one click.

The broader lesson, beyond the specific implementation, is that "MCP versus direct API" is a false dichotomy. What matters is the quality of the documentation, not the protocol. Direct APIs with good schemas often outperform MCP servers with bad ones. We adopted MCP because the standard is converging, not because the protocol is inherently superior. The skills are protocol-agnostic; only the connectors care which transport carries the data.

What broke

The pilot did not produce any catastrophic failures. No hallucinated data published externally. No confidential information leaked. No fundamentally wrong outputs. This is worth noting and not worth overclaiming. Participants were attentive, stakes were managed, and the system itself enforces enough guardrails that the obvious failure modes did not trigger.

What did break, in order of how much operational time it consumed:

Token consumption visibility. The most operationally costly failure mode was that contributors had no real-time feedback on token usage at the session level. A user running long sessions with accumulated context could consume ten times what a user running short focused sessions consumed, without either of them knowing. Token costs scale with accumulated context, not just with the current query. We developed practices documented in the next section. The visibility problem itself is a tooling gap that vendors are working on; for now, managing shared credit pools requires trust rather than data.

Sync drift. The shared repository had three documented incidents where contributors started working on local state that was twelve commits behind the remote, including their own commits made from a different machine. State drift was silent until verified manually. We added a session-start convention to the workspace manifest: on the first turn of any new session, the agent runs `git fetch` and reports if behind. This catches drift before any work begins. It does not eliminate the problem; it surfaces it.

FUSE mount and Git operations. The desktop interface mounts the workspace through a FUSE filesystem that does not support all the file operations Git requires. Specifically, `.git/index.lock` and similar lock files cannot be created or deleted reliably inside the sandbox. Any Git write operation (commit, merge, rebase) is structurally impossible from inside the desktop interface and must happen from a native terminal. We documented the workaround (clone on the host filesystem, edit through the desktop interface, commit from terminal) but the friction recurred for every contributor at some point. This is a vendor-side limitation that will likely be resolved by tooling improvements.

Git as the toll gate. For contributors with an engineering background, branching, committing, and pulling are baseline reflexes. For PMs, marketing, customer success, and operations, they are not. Every non-engineer in the pilot hit the same wall in week one: not the agent, not the substrate, but the version control workflow underneath. They learned it, and the agent helped — natural-language Git commands, narrated diffs, branch-naming suggestions, all of which reduced the cliff — but the cliff was real. This is the most universal friction we observed for new contributors today, and the one we expect

tooling to absorb fastest. Until it does, plan for it. The thesis "anyone can become an Architect" stands only with the caveat that the Architect still has to put their work into Git.

Schema ambiguity. Before schema-level prompt engineering documentation landed, the same query against the same data could return different results across sessions because the model formulated the query slightly differently each time. The fix reduced variance significantly but did not eliminate it. Deterministic outputs across sessions are a work in progress and will remain one for a while.

Claude refusing to execute on ethical grounds. A contributor built a skill for OSINT (open source intelligence) collection. The agent questioned the skill's purpose before executing, requesting more context about the legitimate use case. The first occurrence was startling. Once the contributor framed the context properly in the skill's metadata, the agent proceeded. This is governance the architecture did not write — the model itself enforces a layer of ethical scrutiny that the team did not anticipate. It can be feature (automatic guardrail) or friction (blocker for legitimate work). Both have happened. It is a property of the architecture that operators of agentic systems have to internalize: the model is not a neutral executor.

Vendor downtime. The agent platform itself had at least one prolonged outage during the pilot that interrupted productive work for hours. When the team chat registered "I cannot work right now" in response to vendor 500 errors, the system had crossed the threshold from auxiliary tool to operational dependency. This is an honest cost of deep adoption. SLAs and fallback strategies become real considerations.

Live Artifact token amplification. A contributor built an HTML "Live Artifact" that reloaded fresh data from multiple MCPs every time the page opened. The pattern is powerful (snapshot becomes recurring panel) but it amplifies token cost. Each open is multiple MCP queries. Heavy usage of Live Artifacts can consume significant credit. Worth flagging as a pattern that scales differently than static artifacts.

Inconsistencies caught by audits, not by the system. Files and paths drifted out of sync across contributors without the system surfacing the inconsistency. The agent caught the inconsistencies when explicitly asked to audit, but did not enforce sync discipline on its own. Periodic agent-assisted audits became part of maintenance.

The pattern across all of these: the architecture works under pressure. The friction is real and worth budgeting time for. None of it is a deal-breaker, but pretending it does not exist would mislead anyone trying to repeat this.

Tokens are the new working capital

If you take one operational lesson from this document, take this one. Token consumption changes how the team operates, and it is the failure mode most contributors hit first.

The mechanism: an instruction's token cost scales with the accumulated context of the session. An instruction that costs 20 tokens in isolation may cost 5,000 tokens within a session that has accumulated 10,000 tokens of prior context, because the model processes the new instruction together with the full history. Running trivial tasks within a long session is disproportionately expensive relative to running the same task in a fresh one.

This was not obvious at the start of the pilot. Contributors who consumed the most tokens were often unaware they were doing so. Without per-user visibility, individuals had no way to self-correct.

Session hygiene practices that emerged:

- **Reset context between distinct tasks.** A new task starts a new session. Old context is not carried over.
- **Compact long sessions before switching focus.** If a session has accumulated significant context, end it cleanly (let the agent summarize what was done into memory files) and start fresh for the next thing.

- **Specify exact file paths instead of asking the agent to search.** Search consumes tokens; explicit pointers do not.
- **Generate scripts for repetitive tasks and run them externally.** A Python script that runs 200 times is cheaper than 200 agent calls.
- **Save learned context to the knowledge base at session end.** Future sessions load it rather than rediscover it.
- **Match model to task.** A higher-capability model is appropriate for planning and architecture work. A faster, lower-cost model handles execution and recurring workflows without quality degradation. This choice is not obvious to most users: pricing models are opaque, the default is usually the most expensive option, and most contributors do not realize that the cheaper tier handles the majority of their tasks at the same quality. Surface the choice to the team explicitly, early.

Tokens are the new working capital. The general lesson for any team running this architecture is that token consumption is a budget problem, not just a usage problem. It requires the same kind of attention that AWS bills required when cloud started, and tools for visibility are still maturing. Plan for it.

What we don't yet know

The pilot is seven weeks old. Some questions are answerable from the data we have. Some we do not yet know.

Sustainability. Will the knowledge base stay current over months? At seven weeks, contributors are still in the productive enthusiasm phase. The harder test is whether the substrate maintains itself when the novelty wears off. There is precedent for trying without a substrate underneath: HubSpot eliminated org charts and titles around 2013–2014 and reverted after nine months because the coordination cost without a system to carry it was unsustainable. Our bet is that the substrate is the piece that was missing then. The next twelve months will tell us whether the bet holds.

Governance at scale. With twenty contributors, conventions like branch-per-default are sufficient. With two hundred, they will not be. We do not yet know what additional governance the architecture needs. Likely candidates include automated content review, ownership signals per knowledge area, schemas for contribution types, and possibly a lightweight CI pipeline that runs consistency checks on the knowledge base before merge.

The role of humans versus agents. The most consequential question is also the most open. As agents become more capable, where the line moves matters more. The pilot's honest answer is that we have data on what changed in seven weeks, but we cannot project what changes in seven months or seven quarters. Some functions absorbed agent capability faster than expected. Others are still figuring out where they end and the agent begins. This is not a problem to solve once. It will be ongoing renegotiation.

Onboarding at scale. The setup-interview pattern handled the first twenty contributors well. It is not obvious it scales to two hundred without modification. The question is not whether the technical onboarding works (it does); it is whether the cultural onboarding — the understanding of why the architecture matters, why conventions exist, what the substrate is for — scales without explicit teaching at every step.

How sensitive content scales. The structure we use today — one shared repository, project folders for relevance — works for the substrate at our current size and content mix. It does not have an answer for content that needs hard segregation: financial planning, HR records, M&A material, customer data with regulatory constraints. Project folders give coarse separation by relevance, but the access boundary in our current setup is the repository, not the folder. The most likely path forward is multiple repositories, each with its own access boundary, each with its own substrate. The cost of that path is unclear. Cross-repository references would need to resolve cleanly. The agent would need to operate across multiple substrates without leaking context from one to another. Knowledge that sits in both worlds — a glossary, a style guide, a brand voice — would either need to duplicate or to live in a public reference both can read. We have not chosen between

those tradeoffs. This is the first governance question we do not yet have a workable answer for, and the gap matters more as the substrate grows.

These questions do not block the experiment. They define the next phases.

If you want to try this

The setup is smaller than the discourse around agentic AI suggests. The minimum requirements:

Tools.

- A Git hosting service for the shared repository (GitHub, GitLab, Bitbucket).
- A local Git client for each participant.
- An LLM agent that can read files and create documents (Claude via Claude Code or Cwork, Cursor, GitHub Copilot, OpenCode, AntiGravity, or similar).
- A workspace folder on each participant's machine where the agent operates.

First files, in order.

1. Create the shared repository. Add `knowledge-base/`, `shared-projects/`, `tools/` as empty directories. Add a top-level `README.md` that explains the layout.
2. In each participant's workspace, add `CLAUDE.md` (or equivalent for your tool), `TASKS.md`, and a `memory/` directory. All three are personal and should be `.gitignored` from the shared repo.
3. Populate `CLAUDE.md` with a thin manifest (about fifty lines) that points the agent to the shared and personal layers, defines status markers, and lists basic preferences. Use the starter kit template if helpful.

First conventions to adopt.

- Status markers for unverified or in-progress content (`[ TBD ]`, `[ NEEDS-UPDATE ]`, `[ DRAFT ]`).
- Branch-per-default for non-trivial changes to the shared knowledge base.
- Session-start `git fetch` as the first command of any new session.
- Session-end review that updates personal memory files based on what happened.

First 24 hours.

1. Populate `knowledge-base/` with one or two domain files relevant to your team. Product descriptions, key processes, glossary of internal terms — whatever is most asked-about and most often outdated. If you have legacy material in PDFs or slide decks, convert it to Markdown first and ask the agent to distill the result before committing. The distillation is its own quality gate: anything that does not survive the agent's summary did not belong in the substrate to begin with.
2. Use the agent for a real task. Anything: drafting a stakeholder update, summarizing a meeting, comparing two product capabilities, preparing for a customer call. Watch what the agent gets right and what it gets wrong.
3. Update the knowledge base based on what you learned. The first failure modes are also the first signals of what is missing.

Common pitfalls to avoid.

- Putting too much in the instruction manifest. Keep it around fifty lines, push everything else to dedicated files.
- Treating the knowledge base as documentation for humans. It is operating context for agents. It should answer "what does the agent need to know to act on my behalf", not "what should a new hire read on day one".
- Underestimating token consumption. Read *Tokens are the new working capital* before scaling usage.
- Trying to onboard everyone in week one. Start with two people, ideally two who already tinker with their own tools — the rest of the team will join after seeing what changes for them. Twenty mandated participants will not pull a function the way two evangelists will.
- Stopping after the first wins. Once your first contributors each have a visible win, design the second phase explicitly: their job shifts from "first builder" to "team puller". That transition does not happen on its own. Name it as a phase, give them ownership of getting their function past individual productivity gains, and define what function-level change looks like for them. The second phase is where the architecture stops being about individual productivity and starts being about how a function operates differently.

The starter kit at [agentic-os-starter-kit](#) contains all of the above as a working seed. Fork it, populate the knowledge base with your own context, and run.

Companion document: the manifesto, [Restructuring Knowledge Work for Human-Agent Teams](#).